AI Loan Eligibility Checker

AI-Powered BFSI Financial Decision Assistant

A modern, production-quality fintech web application that enables users to check loan eligibility, analyze credit scores, calculate EMI, and receive personalized financial guidance powered by artificial intelligence.

🏷 Table of Contents

- Project Overview
- Features
- Technology Stack
- Architecture
- Hardware Requirements
- Software Requirements
- Installation
- Configuration
- Running Locally
- Testing
- Deployment
- API Documentation
- Financial Disclaimer
- Future Enhancements
- Support

🎯 Project Overview

The AI Loan Eligibility Checker is a comprehensive BFSI (Banking, Financial Services & Insurance) web platform designed to:

- **Assess Loan Eligibility** - Check if you meet eligibility criteria based on deterministic financial rules
- **Analyze Credit Scores** - Understand your creditworthiness and get improvement recommendations
- **Calculate EMI** - Compute monthly EMI and total payment using standard financial formulas
- **Provide Financial Guidance** - Receive personalized financial tips from AI advisors

The application combines:

- **Deterministic Rule Engine** - Transparent, auditable eligibility logic
- **AI Integration** - Claude AI and Groq for intelligent financial analysis
- **Cloud Storage** - Google Sheets integration for persistent data
- **Local Persistence** - Browser LocalStorage for offline functionality

---

✨ Features

1. Loan Eligibility Checker (Primary Feature)

- Enter financial information (name, salary, credit score, existing EMI, age)
- Instant eligibility decision based on 4 core rules:
  - Monthly salary > ₹30,000
  - Credit score > 700
  - Existing EMI < ₹20,000
  - Age ≥ 21 years
- Calculate eligible loan amount (Salary × 20)
- View passed/failed conditions with explanations
- Risk classification (Low/Moderate/High)
- AI-powered recommendations

2. Credit Score Analyzer

- Analyze credit scores (300-900 range)
- Automatic classification:
  - 750-900: Excellent
  - 650-749: Good
  - 300-649: Poor
- Visual progress indicator
- Personalized recommendations based on score range

3. EMI Calculator

- Calculate monthly EMI with precision
- Standard financial formula: EMI = (P × R × (1 + R)^N) / ((1 + R)^N – 1)
- Handles zero-interest loans
- Displays:
  - Monthly EMI
  - Total Interest
  - Total Payment
  - Tenure breakdown

4. AI Financial Tips

- Ask financial questions
- Receive personalized guidance from Claude AI
- Fallback to rule-based tips if AI unavailable
- Topics: Improving eligibility, credit scores, EMI management, expense management

5. Analytics Dashboard

- Total eligibility checks
- Eligible vs. not eligible count
- Average credit score
- Average loan amounts
- Statistical insights

6. Analysis History

- View previous eligibility checks
- Delete individual records
- Clear all history
- Local browser persistence

🛠 Technology Stack

Frontend

- **HTML5** - Semantic markup
- **CSS3** - Dark glassmorphism design
- **Vanilla JavaScript** - No framework, pure JS
- **Font Awesome** - Icon library
- **Google Fonts** - Typography (Manrope, Poppins, Space Mono)

Backend

- **Python 3.8+** - Core language
- **Flask 3.0** - Web framework
- **Flask-CORS** - Cross-origin requests
- **Requests** - HTTP library for external APIs

AI & Integrations

- **Claude AI** - Anthropic API for financial analysis (Primary)
- **Groq API** - Alternative fast inference provider
- **Google Apps Script** - Sheets integration for data persistence

Deployment & Tools

- **Git/GitHub** - Version control
- **Ngrok** - Local tunneling for testing
- **Flask Development Server** - Local development
- **Docker** - Optional containerization
- **Heroku/Railway** - Cloud deployment

Architecture

```
ai-loan-eligibility-checker/
│
├── app.py                              # Flask backend (main
├── requirements.txt                    # Python dependencies
├── .env.example                        # Environment configu
├── .env                                # Local environment (
├── .gitignore                          # Git ignore rules
├── README.md                           # Documentation (this
│
├── templates/
│   └── index.html                      # Single-page applica
│
├── static/
│   ├── style.css                       # Dark glassmorphism
│   └── script.js                       # Vanilla JavaScript
│
└── optional/
    ├── Dockerfile                      # Docker containeriza
    ├── docker-compose.yml              # Docker compose conf
    └── .github/workflows/               # GitHub Actions CI/
```

Application Flow

```
User Opens Application
        ↓
Landing Dashboard with Hero Section
```

```
        ↓
Select Financial Tool:
  - Loan Eligibility Checker
  - Credit Score Analyzer
  - EMI Calculator
  - AI Financial Tips
        ↓
Frontend Validation
        ↓
Backend Validation
        ↓
Process Request:
  - Deterministic Rule Engine (Loan Eligibility)
  - Financial Calculations (Credit Score, EMI)
  - AI Analysis (Claude/Groq)
        ↓
Generate Result
        ↓
Save to LocalStorage (Browser)
        ↓
Attempt Save to Google Sheets
        ↓
Display Result to User
        ↓
Update Dashboard & History
```

🖥️ Hardware Requirements

Component	Minimum	Recommended
Processor	Intel Core i5 (8th Gen) / AMD Ryzen 5	Intel Core i7 (10th Gen+) / AMD Ryzen 7
RAM	8 GB	16 GB
Storage	256 GB SSD	512 GB SSD

GPU	Optional	NVIDIA GTX 1650+ (for AI/ ML experiments)
Internet	5 Mbps+	20 Mbps+

📦 Software Requirements

Operating Systems

- **Windows** - Windows 10 / Windows 11
- **macOS** - Monterey (12.0) or later
- **Linux** - Ubuntu 20.04 LTS or later, Debian 11+

Development Tools

- **Python** - 3.8 or higher (Download)
- **Git** - 2.25 or higher (Download)
- **Pip** - Python package manager (comes with Python)
- **Venv** - Python virtual environment (built-in)

Browsers (Testing)

- **Google Chrome** - Latest version
- **Mozilla Firefox** - Latest version
- **Microsoft Edge** - Latest version
- **Safari** - Latest version (macOS)

Cloud Services (Optional)

- **Claude API** - Anthropic AI integration
- **Groq API** - Alternative AI provider
- **Google Apps Script** - Sheets data persistence

- **Ngrok** - Local tunneling for webhook testing

📥 Installation

Step 1: Clone Repository

```
git clone https://github.com/yourusername/ai-loan-eligibi
cd ai-loan-eligibility-checker
```

Step 2: Create Python Virtual Environment

```
# Create virtual environment
python -m venv venv

# Activate virtual environment
# On Windows:
venv\Scripts\activate

# On macOS/Linux:
source venv/bin/activate
```

Step 3: Install Dependencies

```
pip install -r requirements.txt
```

Verify installation:

```
pip list
```

Step 4: Configure Environment Variables

```
# Copy example file
cp .env.example .env

# Edit .env file with your configuration
# See Configuration section below
```

🔑 Configuration

Claude AI Configuration

Get Your API Key:

1. Visit Anthropic Console
2. Sign up or log in
3. Navigate to "API Keys"
4. Click "Create Key"
5. Copy the key (starts with `sk-ant-`)

Configure in .env:

```
AI_PROVIDER=claude
CLAUDE_API_KEY=sk-ant-your-api-key-here
```

Verify Configuration:

```
curl -X GET http://localhost:5000/api/health
```

Expected response:

```
```

```
{
  "status": "healthy",
  "ai_provider": "claude",
  "claude_configured": true,
  "groq_configured": false
}
```

Groq API Configuration (Alternative)

Get Your API Key:

1. Visit Groq Console
2. Sign up or log in
3. Create an API key
4. Copy the key

Configure in .env:

```
AI_PROVIDER=groq
GROQ_API_KEY=your_groq_api_key_here
```

Note: You can have both Claude and Groq configured; Flask will use the active AI_PROVIDER.

Google Sheets Integration

Setup Google Apps Script:

1. **Create a Google Sheet:**
    - Open Google Sheets
    - Create new spreadsheet: "Loan Eligibility Records"
    - Add headers: Timestamp, Applicant Name, Monthly Salary, Credit Score, Existing EMI, Age, Eligibility Status, Risk Classification, Eligible Loan Amount, AI Recommendation

2. **Create Google Apps Script:**

   - Go to Google Apps Script
   - Create new project
   - Replace code with:

```
function doPost(e) {
  const sheet = SpreadsheetApp.getActiveSheet();
  const data = JSON.parse(e.postData.contents);

  sheet.appendRow([
    new Date().toLocaleString(),
    data.name || '',
    data.salary || '',
    data.credit_score || '',
    data.existing_emi || '',
    data.age || '',
    data.eligible ? 'ELIGIBLE' : 'NOT ELIGIBLE',
    data.risk_classification || '',
    data.eligible_loan_amount || '',
    data.ai_recommendation || ''
  ]);

  return ContentService.createTextOutput(
    JSON.stringify({success: true})
  ).setMimeType(ContentService.MimeType.JSON);
}
```

3. **Deploy as Web App:**

   - Click "Deploy" → "New Deployment"
   - Choose "Web app"
   - Execute as: Your account
   - Who has access: "Anyone"
   - Copy the deployment URL

4. **Configure in .env:**

```
GOOGLE_SHEETS_ENDPOINT=https://script.google.com/macros/s
```

Important:

- Google Sheets integration is **optional**
- App works without it using LocalStorage
- If Sheets is not configured, no error occurs
- Failed Sheets saves don't break the application

🚀 Running Locally

Start Flask Server

```
# Make sure virtual environment is activated
source venv/bin/activate  # macOS/Linux
# or
venv\Scripts\activate  # Windows

# Run Flask
python app.py
```

Expected output:

```
 * Serving Flask app 'app'
 * Debug mode: on
 * Running on http://127.0.0.1:5000
 * Press CTRL+C to quit
```

Access Application

Open browser and navigate to:

```
http://localhost:5000
```

Test API Endpoint

```
# Health check
curl http://localhost:5000/api/health

# Test eligibility check
curl -X POST http://localhost:5000/api/eligibility \
  -H "Content-Type: application/json" \
  -d '{
    "name": "John Doe",
    "salary": 50000,
    "credit_score": 750,
    "existing_emi": 10000,
    "age": 25
  }'
```

Testing

Test Scenarios

Loan Eligibility Tests

Test Case 1: Eligible Application

```
Input:
  Salary: ₹50,000
  Credit Score: 750
  Existing EMI: ₹10,000
  Age: 25

Expected:
  ✓ Eligible
  Eligible Loan Amount: ₹10,00,000
```

```
  Risk: LOW RISK
```

Test Case 2: Insufficient Salary

```
Input:
  Salary: ₹25,000
  Credit Score: 750
  Existing EMI: ₹10,000
  Age: 25

Expected:
  ✗ Not Eligible
  Reason: Salary < ₹30,000
```

Test Case 3: Low Credit Score

```
Input:
  Salary: ₹50,000
  Credit Score: 680
  Existing EMI: ₹10,000
  Age: 25

Expected:
  ✗ Not Eligible
  Reason: Credit Score < 700
```

Test Case 4: High EMI Obligations

```
Input:
  Salary: ₹50,000
  Credit Score: 750
  Existing EMI: ₹25,000
  Age: 25

Expected:
  ✗ Not Eligible
  Reason: Existing EMI ≥ ₹20,000
```

Test Case 5: Below Minimum Age

```
Input:
  Salary: ₹50,000
  Credit Score: 750
  Existing EMI: ₹10,000
  Age: 20

Expected:
  ✗ Not Eligible
  Reason: Age < 21
```

Credit Score Tests

Score	Category	Result
300	Poor	✓ Correct
500	Poor	✓ Correct
649	Poor	✓ Correct
650	Good	✓ Correct
700	Good	✓ Correct
749	Good	✓ Correct
750	Excellent	✓ Correct
850	Excellent	✓ Correct
900	Excellent	✓ Correct

EMI Calculator Tests

Test: Standard EMI

```
Input:
  Principal: ₹10,00,000
  Interest Rate: 8%
  Tenure: 60 months
```

```
Expected:
  Monthly EMI: ₹20,276 (approximately)
  Total Interest: ₹2,16,560
  Total Payment: ₹12,16,560
```

Test: Zero Interest

```
Input:
  Principal: ₹10,00,000
  Interest Rate: 0%
  Tenure: 60 months

Expected:
  Monthly EMI: ₹16,666.67
  Total Interest: ₹0
  Total Payment: ₹10,00,000
```

Running Automated Tests

```
# Run pytest (if implemented)
pytest tests/

# Run with coverage
pytest --cov=. tests/
```

Manual Testing Checklist

- ☐ Loan Eligibility Form validates all fields
- ☐ Eligibility engine calculates correctly
- ☐ Credit Score Analyzer displays correct categories
- ☐ EMI Calculator uses correct formula
- ☐ AI tips generate successfully
- ☐ History saves to LocalStorage
- ☐ Delete history record works
- ☐ Clear all history works

- ☐ Dashboard updates correctly
- ☐ Mobile layout responsive
- ☐ No console errors
- ☐ All buttons functional
- ☐ Forms reset after submission
- ☐ Error messages display correctly
- ☐ Toast notifications appear
- ☐ Loading states show
- ☐ API health check responds

🌐 Deployment

Deploying to Heroku

Prerequisites:

- Heroku account (Sign up)
- Heroku CLI (Install)

Steps:

1. **Create Heroku App:**

```
heroku create your-app-name
```

2. **Add Buildpacks:**

```
heroku buildpacks:set heroku/python
```

3. **Create Procfile:**

```
echo "web: python app.py" > Procfile
```

4. **Set Environment Variables:**

```
heroku config:set CLAUDE_API_KEY=your_key
heroku config:set AI_PROVIDER=claude
heroku config:set FLASK_ENV=production
```

5. **Deploy:**

```
git push heroku main
```

6. **Access App:**

```
heroku open
```

Deploying to Railway

1. **Connect Repository:**
    - Go to Railway.app
    - Link your GitHub repository
    - Select your project
2. **Configure Environment Variables:**
    - Add CLAUDE_API_KEY, AI_PROVIDER, etc.
3. **Deploy:**
    - Railway auto-deploys on push

Deploying to PythonAnywhere

1. **Create Account:** PythonAnywhere.com
2. **Upload Files:** Via Git or web interface
3. **Configure:**
    - Set Python version to 3.9+
    - Configure WSGI file

- Set environment variables

4. **Deploy:** Click "Reload"

Using Ngrok for Local Testing

Expose your local server to the internet:

```
# Install Ngrok
# Download from https://ngrok.com/download

# Run Ngrok
ngrok http 5000

# You'll get a URL like: https://abcd1234.ngrok.io
# Use this URL to test webhooks, integrations, etc.
```

API Documentation

Health Check

Endpoint: `GET /api/health`

Response:

```
{
  "status": "healthy",
  "timestamp": "2024-01-15T10:30:00",
  "ai_provider": "claude",
  "claude_configured": true,
  "groq_configured": false,
  "google_sheets_configured": false
}
```

Check Loan Eligibility

Endpoint: `POST /api/eligibility`

Request:

```
{
  "name": "Vinay Kumar",
  "salary": 50000,
  "credit_score": 750,
  "existing_emi": 12000,
  "age": 22
}
```

Response (Eligible):

```
{
  "success": true,
  "applicant_name": "Vinay Kumar",
  "eligible": true,
  "eligible_loan_amount": 1000000,
  "risk_classification": "LOW RISK",
  "conditions_passed": [...],
  "conditions_failed": [],
  "ai_recommendation": "..."
}
```

Analyze Credit Score

Endpoint: `POST /api/credit-score/analyze`

Request:

```
{
  "credit_score": 750
}
```

Response:

```
{
  "success": true,
  "credit_score": 750,
  "category": "EXCELLENT",
  "description": "Outstanding credit profile...",
  "recommendations": [...]
}
```

Calculate EMI

Endpoint: `POST /api/emi/calculate`

Request:

```
{
  "principal": 1000000,
  "annual_rate": 8.5,
  "tenure_months": 60
}
```

Response:

```
{
  "success": true,
  "principal": 1000000,
  "annual_rate": 8.5,
  "tenure_months": 60,
  "monthly_emi": 20276,
  "total_interest": 216560,
  "total_payment": 1216560
}
```

Get Financial Tips

Endpoint: `POST /api/ai/tips`

Request:

```
{
  "question": "How can I improve my loan eligibility?"
}
```

Response:

```
{
  "success": true,
  "tip": "Focus on three key areas...",
  "from_ai": true
}
```

🔐 Security Considerations

API Key Management

- ✓ API keys stored in `.env` (not committed)
- ✓ Backend handles all AI API calls
- ✓ Frontend never accesses API keys
- ✓ CORS configured for production

Data Privacy

- ✓ No sensitive data stored permanently
- ✓ LocalStorage used for user data only
- ✓ HTTPS recommended for production
- ✓ User data not shared with third parties

Best Practices

1. **Never commit `.env` file**
2. **Rotate API keys regularly**
3. **Use strong passwords**
4. **Enable CORS only for trusted origins**
5. **Validate all input on backend**
6. **Use HTTPS in production**

⚠ Financial Disclaimer

IMPORTANT: This tool provides an educational financial assessment based on the information entered.

It does NOT:

- Guarantee loan approval
- Replace official bank underwriting
- Provide legal or professional financial advice
- Constitute an offer of credit
- Guarantee any specific interest rates
- Consider all factors a bank would consider

Users should:

- Consult qualified financial advisors
- Verify information with actual lenders
- Review terms and conditions carefully
- Understand risks of financial products
- Not rely solely on this tool for major financial decisions

The application is provided "as-is" without warranties or guarantees.

Future Enhancements

Phase 2 Features

- ☐ Multi-language support
- ☐ Dark/Light theme toggle
- ☐ Real-time interest rate lookup
- ☐ Loan comparison tool
- ☐ Portfolio analysis
- ☐ Financial goal planning
- ☐ Budget management module

Phase 3 - Advanced

- ☐ Machine learning model for better predictions
- ☐ Integration with actual lending platforms
- ☐ Real-time credit score updates
- ☐ Automated loan applications
- ☐ Document upload and verification
- ☐ Mobile app (React Native)

Infrastructure

- ☐ Docker containerization
- ☐ Kubernetes deployment
- ☐ CI/CD pipeline (GitHub Actions)
- ☐ Automated testing
- ☐ Performance monitoring
- ☐ Error tracking (Sentry)
- ☐ Analytics (Mixpanel)

📧 Support

Getting Help

Documentation:

- Read this README thoroughly
- Check API Documentation section
- Review Configuration guides

Issues:

- GitHub Issues: GitHub
- Email: support@example.com

Contributing:

- Fork the repository
- Create feature branch
- Submit pull request
- Follow code style

License

This project is licensed under the MIT License - see LICENSE file for details.

Acknowledgments

- **Claude AI** - Anthropic for advanced financial analysis
- **Groq** - Fast inference API
- **Google** - Apps Script and Sheets integration
- **Flask Community** - Amazing web framework
- **Open Source** - Built on open-source tools

Project Statistics

- **Lines of Code:** ~4,000+
- **CSS Styles:** ~1,500+
- **JavaScript Functions:** 40+
- **API Endpoints:** 5
- **Supported AI Providers:** 2
- **UI Components:** 15+
- **Responsive Breakpoints:** 3

🎓 Learning Resources

- Flask Documentation
- Claude API Guide
- JavaScript MDN Docs
- CSS Tricks
- Google Apps Script Docs

Last Updated: January 2024 **Version:** 1.0.0 **Maintained By:** Your Name / Organization